

1. Javascript Questions on Variables, Operators, and If-Else** from **basic to advanced**:

Basic Level

1. What is the difference between `var`, `let`, and `const` in JavaScript?
2. How do you declare a variable in JavaScript, and what are the naming conventions?
3. What are the different types of operators in JavaScript?
4. What is the difference between `==` and `===` operators?
5. How do arithmetic operators (+, -, *, /, %) work in JavaScript?
6. What will be the output of `console.log(5 + "5")`? Explain why.
7. How does the `typeof` operator work in JavaScript?
8. How do logical operators (&&, ||, !) work? Provide an example.
9. What is the purpose of the **ternary operator** in JavaScript?
10. How do comparison operators (>, <, >=, <=) work in JavaScript?
11. Declare a variable `name` and assign it your name. Print it to the console.
12. Create two variables `a = 10` and `b = 5`, then print their sum.
13. Write a JavaScript program to check whether a given number is positive or negative using `if-else`.
14. Declare a variable `age`. If `age` is greater than or equal to 18, print **"You are an adult"**, otherwise print **"You are a minor"**.
15. Write a program that swaps the values of two variables without using a third variable.
16. Declare a variable `isSunny` and set it to `true`. Use an `if` statement to print **"Wear sunglasses"** if `isSunny` is `true`.
17. Create a variable `x = "10"`. Convert it into a number and add 5 to it. Print the result.
18. Write a program that checks if a number is **divisible by 5** and prints **"Divisible"** or **"Not Divisible"**.
19. Write a program that checks whether a given year is a **leap year** using `if-else`.
20. Use a ternary operator to check if a number is even or odd.

Intermediate Level

1. What is the difference between **post-increment** (`x++`) and **pre-increment** (`++x`) operators?
2. Explain how assignment operators (`+=`, `-=`, `*=`, `/=`) work in JavaScript.
3. What will be the output of `console.log(null == undefined)`; and why?
4. How does JavaScript handle implicit type conversion (type coercion)? Give an example.
5. Write a JavaScript program that checks if a number is **even or odd** using `if-else`.
6. How can we use multiple conditions inside an `if-else` statement? Provide an example.
7. Explain **falsy** and **truthy** values in JavaScript with examples.
8. What happens if you compare a string with a number using `==` and `===`?
9. Write a program that takes three numbers as input and prints the **largest** of them.

10. Write a JavaScript program that checks whether a person is eligible to vote based on their `age` and `citizenship`.
11. Given `x = 5`, predict the output of `console.log(x++ * 2)`; and explain why.
12. Write a program that takes a number input from the user and prints "**Positive**", "**Negative**", or "**Zero**" based on its value.
13. Using logical operators, check if a person is eligible for a job if they have **5+ years of experience** and a **degree**.
14. Write a JavaScript program that checks whether a given character is a vowel or consonant using `if-else`.
15. Write a program that calculates and prints the **discount amount** based on the following conditions:
 - If the total amount is more than 5000, apply a 10% discount.
 - Otherwise, apply a 5% discount.
16. Write a JavaScript program that determines the **grade** of a student based on marks:
 - `>=90` → "A"
 - `>=80` → "B"
 - `>=70` → "C"
 - `>=60` → "D"
 - `<60` → "F"

Advanced Level

1. How does JavaScript handle **short-circuit evaluation** with `&&` and `||` operators?
 2. Write a JavaScript program that takes **two numbers as input** and determines the **largest** using `if-else`.
 3. Write a program that takes a number input and prints "**Fizz**" if it is divisible by 3, "**Buzz**" if divisible by 5, and "**FizzBuzz**" if divisible by both.
 4. Write a program to compare two numbers and print "Both are equal", "First is greater", or "Second is greater" using a **nested if-else** statement.
-

2. Javascript Questions Using Only `switch`

1. Write a JavaScript program that takes a **day number (1-7)** and prints the corresponding **day name** using `switch`.
 2. Create a program that asks the user to enter a **color name** and uses a `switch` statement to print:
 - "Red means Stop"
 - "Green means Go"
 - "Yellow means Slow Down"
 - "Other means Unknown Color"
 3. Write a program that takes a number (1-12) and prints the corresponding **month name** using `switch`.
 4. Develop a simple calculator using `switch` that takes two numbers and an operator (+, -, *, /) as input and prints the result.
-

3. Javascript Questions Containing Variables, If-Else, Switch, and Operators

1. Write a program that asks the user to enter a **number** and checks whether it is **positive, negative, or zero** using `if-else`. If it is zero, use a `switch` statement to print "Number is zero".
 2. Create a program that takes **two numbers and an operator (+, -, *, /)**. Use `if-else` to check for valid inputs and `switch` to perform the calculation.
 3. Write a program that takes a **student's marks** and prints the **grade** using `if-else`. Also, use a `switch` statement to print remarks like "Excellent", "Good", or "Needs Improvement".
 4. Take a **year as input** and use `if-else` to check if it's a **leap year**. If it is, use `switch` to print "Leap Year"; otherwise, print "Not a Leap Year".
 5. Create a program that asks the user to enter a **month number (1-12)**. Use `if-else` to check if it's a valid month, then use `switch` to print the season (Winter, Summer, Monsoon, etc.).
 6. Write a program that takes an **age** as input and uses `if-else` to check if a person is **eligible to drive**. If they are under 18, use `switch` to suggest "Wait until you turn 18" or "Apply for a learning license".
 7. Develop a program that asks the user to enter a **vehicle type (car, bike, truck)**. Use `if-else` to check if the input is valid, then use `switch` to print the **road tax percentage** based on the vehicle type.
 8. Write a program that asks the user to enter a **number between 1-10**. If the number is **even**, use `switch` to print "Even Number", otherwise, print "Odd Number".
 9. Create a **temperature converter** program that takes the temperature in **Celsius or Fahrenheit** as input. Use `if-else` to determine the input type and `switch` to convert it.
 10. Write a program that takes a **fruit name** as input and uses `if-else` to check if it's valid. Then, use `switch` to print whether it's a **citrus fruit (like Orange, Lemon)** or a **non-citrus fruit (like Apple, Banana)**.
-

4. JavaScript questions using for loops, while loops, and do-while loops

Basic Level (Using Loops + Variables + Operators + If-Else)

1. **Print Numbers 1 to 10:** Write a `for` loop to print numbers from 1 to 10.
2. **Sum of First N Natural Numbers:** Write a program that takes a number `n` as input and prints the sum of the first `n` natural numbers using a `while` loop.
3. **Even Numbers from 1 to 50:** Use a `for` loop to print all even numbers from 1 to 50.
4. **Reverse Counting:** Write a program to print numbers from 10 to 1 using a `do-while` loop.
5. **Factorial Calculation:** Write a program that takes a number as input and calculates its factorial using a `for` loop.
6. **Multiplication Table:** Ask the user for a number and print its multiplication table using a `for` loop.
7. **Sum of Even & Odd Numbers Separately:** Use a `while` loop to find the sum of even and odd numbers from 1 to 100 separately.
8. **Check Prime Number:** Write a program that takes a number as input and checks whether it is prime or not using a `for` loop.
9. **Reverse a Number:** Use a `while` loop to reverse a given number (e.g., input 1234, output 4321).
10. **Check Palindrome Number:** Take a number as input and check if it is a palindrome using a `while` loop (e.g., 121 is a palindrome).

11. **Print Odd Numbers 1 to 50:** Use a `for` loop to print all odd numbers from 1 to 50.
12. **Print Square of First 10 Numbers:** Write a `for` loop to print the square of numbers from 1 to 10.
13. **Reverse a String:** Take a string as input and reverse it using a `for` loop.
14. **Check if a Number is a Perfect Number:** A perfect number is a number whose sum of divisors (excluding itself) equals the number (e.g., $6 = 1+2+3$). Use a `for` loop to check for a given number.
15. **Print Numbers in Steps:** Use a `for` loop to print numbers from 1 to 100 in steps of 5 (e.g., 1, 6, 11, 16, ...).
16. **Find the Power of a Number:** Write a program to calculate x^y (x raised to the power y) using a loop instead of `Math.pow()`.
17. **Calculate GCD of Two Numbers:** Write a program that finds the greatest common divisor (GCD) of two numbers using a `while` loop.
18. **Find LCM of Two Numbers:** Take two numbers as input and find their least common multiple (LCM) using a `while` loop.
19. **Find All Factors of a Number:** Use a `for` loop to print all factors of a given number.
20. **Count the Number of Words in a String:** Take a sentence as input and count the number of words using a `for` loop.

Intermediate Level (Using Loops + If-Else + Switch + Conditions)

1. **FizzBuzz Program:** Print numbers from 1 to 50. If a number is divisible by 3, print "Fizz", if divisible by 5, print "Buzz", and if divisible by both, print "FizzBuzz".
2. **Armstrong Number Check:** Write a program to check whether a given number is an Armstrong number (e.g., $153 = 1^3 + 5^3 + 3^3$).
3. **Print Fibonacci Series:** Print the first `n` numbers in the Fibonacci sequence using a `for` loop.
4. **Count Digits in a Number:** Take a number as input and count the number of digits using a `while` loop.
5. **Check Leap Year for Multiple Years:** Take a starting and ending year as input and print all leap years in that range using a `for` loop.
6. **Simple Calculator Using Loops & Switch:** Keep asking the user for two numbers and an operation (+, -, *, /). Use a `do-while` loop to continue until the user exits.
7. **Guess the Number Game:** Generate a random number between 1 and 100. Ask the user to guess it using a `while` loop, providing hints (Too high, Too low) until they guess correctly.
8. **Sum of Digits of a Number:** Take a number as input and calculate the sum of its digits using a `while` loop.
9. **Find Largest & Smallest Number in an Array:** Take an array of numbers and use a `for` loop to find the largest and smallest values.
10. **Count Vowels & Consonants in a String:** Take a string input and count the number of vowels and consonants using a `for` loop.
11. **Generate Random Numbers Until a Condition is Met:** Write a program that generates random numbers between 1-100 until the number 50 is generated. Use a `while` loop.
12. **Check for Strong Number:** A strong number is a number whose sum of the factorial of its digits equals the number itself (e.g., $145 = 1! + 4! + 5!$). Check this using a loop.
13. **Reverse an Array Without Using `.reverse()`:** Take an array and reverse it using a `for` loop.
14. **Find Prime Numbers Between 1 and N:** Take an input `n` and print all prime numbers between 1 and `n` using nested `for` loops.

15. **Find Armstrong Numbers in a Range:** Write a program to print all Armstrong numbers between 1 and 1000 using a `for` loop.
16. **Print a Pattern (Triangle of Stars):** Print the following pattern using nested loops:

```
*
* *
* * *
* * * *
* * * * *
```

17. **Check If a Number is a Palindrome Without String Methods:** Take a number as input and check if it's a palindrome using a `while` loop.
18. **Find the Sum of a Given Range of Numbers:** Take two numbers `a` and `b` as input and sum all numbers between them using a `for` loop.
19. **Simulate a Login System:** Ask the user for a password using a `do-while` loop. Keep asking until the correct password is entered.
20. **Count Occurrences of a Character in a String:** Take a string and a character as input, and count how many times that character appears in the string using a `for` loop.
-

5. JavaScript Object questions

Basic Level (Objects + Variables + Operators + If-Else + Loops)

1. **Create a Student Object:** Create an object `student` with properties: `name`, `age`, `marks`. Print the object.
2. **Access Object Properties:** Given an object `person = { name: "John", age: 30, city: "Delhi" }`, write a program to access and print the `name` property.
3. **Modify Object Properties:** Update the `age` property of the `person` object from the previous question.
4. **Delete a Property from an Object:** Remove the `city` property from the `person` object and print the updated object.
5. **Check if a Property Exists in an Object:** Write a program to check if the `marks` property exists in a given `student` object.
6. **Iterate Over an Object Using a Loop:** Use a `for...in` loop to print all properties and values of an object.
7. **Count the Number of Properties in an Object:** Write a function that counts and returns the number of properties in an object.
8. **Add a Method to an Object:** Create an object `car` with properties like `brand`, `model`, and `year`, and add a method `getDetails()` that returns a string with all details.

9. **Check Object Type:** Write a program that checks if a given variable is an object or not.
10. **Convert Object to Array:** Convert an object into an array of key-value pairs using `Object.entries()`.
11. **Create an Employee Object:** Define an object `employee` with properties: `name`, `age`, `salary`, and `department`. Print the object.
12. **Dynamically Add a Property to an Object:** Add a `gender` property to the `employee` object and print the updated object.
13. **Check If an Object Has a Specific Key:** Write a function that takes an object and a key, and returns `true` if the key exists, otherwise `false`.
14. **Use Object.keys():** Given an object, return an array containing all its keys using `Object.keys()`.
15. **Use Object.values():** Given an object, return an array containing all its values using `Object.values()`.
16. **Use Object.entries():** Convert an object into an array of `[key, value]` pairs using `Object.entries()`.
17. **Loop Through an Object and Print Key-Value Pairs:** Use a `for...in` loop to iterate through an object and print all keys and values.
18. **Freeze an Object:** Use `Object.freeze()` to prevent modifications to an object, then try modifying it.
19. **Seal an Object:** Use `Object.seal()` on an object, then try adding and modifying properties.
20. **Check If Two Objects Are Equal:** Write a function that compares two objects and returns `true` if they have the same properties and values.

Intermediate Level (Objects + Loops + Conditions + Operators + Switch)

1. **Nested Objects:** Create an object `company` with a nested object `address` containing `city`, `state`, and `pincode`. Print the city name.
2. **Merge Two Objects:** Write a function that merges two objects into a single object.
3. **Object with an Array Property:** Create an object `library` that has an array of `books`, each containing `title` and `author`. Print all book titles.
4. **Find an Object in an Array of Objects:** Given an array of `employees`, write a function to find an employee by `id`.
5. **Filter Objects Based on Condition:** Given an array of `students`, filter those who have marks greater than 80.
6. **Sort an Array of Objects:** Given an array of `products`, sort them by `price` in ascending order.
7. **Use a Switch Statement in an Object:** Create an object `calculator` with a method that performs `+`, `-`, `*`, `/` operations based on a `switch` statement.

8. **Check for Empty Object:** Write a function that checks whether an object is empty (has no properties).
 9. **Clone an Object Without Reference:** Write a function that creates a deep copy of an object.
 10. **Convert Object to JSON and Back:** Convert an object into a JSON string using `JSON.stringify()` and parse it back into an object using `JSON.parse()`.
 11. **Nested Objects & Accessing Values:** Create an object `university` with a nested `departments` object containing multiple departments. Access one department's name.
 12. **Object with a Function Property:** Create an object `user` with properties `firstName`, `lastName`, and a method `getFullName()` that returns the full name.
 13. **Convert an Object to an Array of Keys and Values:** Given an object, return an array containing key-value pairs without using `Object.entries()`.
 14. **Find a Key with the Highest Value:** Given an object of student names and marks, return the name of the student with the highest marks.
 15. **Calculate Total Salary of Employees:** Given an array of employee objects with salaries, calculate the total salary using a loop.
 16. **Use Switch Case with an Object:** Create an object `settings` with different modes (`light`, `dark`, `auto`). Use a `switch` statement to print messages based on the selected mode.
 17. **Find the Most Frequent Value in an Object:** Given an object containing repeated values, find the value that appears most frequently.
 18. **Remove Properties with Null or Undefined Values:** Write a function that removes all properties from an object that have `null` or `undefined` values.
 19. **Flatten a Nested Object:** Convert a deeply nested object into a flat object with dot-separated keys.
 20. **Group Objects by a Property:** Given an array of objects (students with departments), group them by department.
-

6. JavaScript Array questions

Basic Level (Arrays + Variables + Operators + If-Else + Loops)

1. **Create an Array and Print Elements:** Create an array of 5 numbers and print all elements using a loop.
2. **Find the Largest Number in an Array:** Write a function that takes an array of numbers and returns the largest number.

3. **Find the Smallest Number in an Array:** Write a function that takes an array of numbers and returns the smallest number.
4. **Reverse an Array Without Using `.reverse()`:** Write a program to reverse an array manually using a loop.
5. **Sum of All Elements in an Array:** Write a function that returns the sum of all numbers in an array.
6. **Filter Even Numbers from an Array:** Write a function that takes an array of numbers and returns only even numbers.
7. **Check If an Array Contains a Specific Element:** Write a function that checks if a given number exists in an array.
8. **Merge Two Arrays Without Duplicates:** Given two arrays, merge them into a single array without duplicate values.
9. **Find the Index of an Element:** Write a function that finds the index of a given element in an array.
10. **Remove Duplicates from an Array:** Write a function that removes duplicate elements from an array.
11. **Check if a Variable is an Array:** Write a function that checks whether a given variable is an array or not.
12. **Convert a String to an Array:** Given a string "Hello World", convert it into an array of words.
13. **Find the Length of an Array:** Write a function that returns the length of an array without using `.length`.
14. **Remove a Specific Element from an Array:** Write a function that removes all occurrences of a given value from an array.
15. **Split an Array into Two Halves:** Given an array of even length, split it into two equal halves.
16. **Find the Difference Between Two Arrays:** Given two arrays, return an array containing elements present in the first but not in the second.
17. **Concatenate Two Arrays Without Using `.concat()`:** Merge two arrays without using `.concat()`.
18. **Find the Most Frequent Element in an Array:** Given an array, determine the element that appears the most times.
19. **Create an Array of N Random Numbers:** Write a function that generates an array of `N` random numbers.
20. **Replace Negative Numbers in an Array with Zero:** Write a function that replaces all negative numbers in an array with 0.
21. **Access the First and Last Element of an Array:** Write a function that returns the first and last elements of an array.
22. **Count the Number of Times a Value Appears in an Array:** Write a function that counts occurrences of a given number in an array.

23. **Reverse an Array Using Recursion:** Implement a function that reverses an array using recursion instead of loops.
 24. **Remove the First and Last Element from an Array:** Write a function that removes the first and last elements from an array and returns the modified array.
 25. **Find the Sum of Alternate Elements in an Array:** Given an array, find the sum of elements at even indices.
 26. **Find the Longest String in an Array:** Write a function that takes an array of strings and returns the longest string.
 27. **Check If Two Arrays Are Identical:** Write a function that checks whether two arrays are identical (have the same elements in the same order).
 28. **Count Positive, Negative, and Zero Values in an Array:** Given an array of numbers, return an object with counts of positive numbers, negative numbers, and zero values.
 29. **Extract the First N Elements of an Array:** Write a function that takes an array and a number N and returns the first N elements.
 30. **Shuffle an Array Randomly:** Implement a function that shuffles the elements of an array randomly.
-

Intermediate Level (Arrays + Loops + Conditions + Objects + Switch)

1. **Sort an Array in Ascending and Descending Order:** Given an array of numbers, sort it in ascending and descending order.
2. **Find the Second Largest Number in an Array:** Write a function that finds the second-largest number in an array without sorting it.
3. **Find Intersection of Two Arrays:** Given two arrays, return a new array containing only the common elements.
4. **Find Union of Two Arrays:** Given two arrays, return a new array containing all elements from both, without duplicates.
5. **Use `.map()` to Double the Values of an Array:** Given an array, return a new array with all elements doubled.
6. **Use `.reduce()` to Find the Total Sum of an Array:** Use `.reduce()` to calculate the sum of an array of numbers.
7. **Find the Frequency of Elements in an Array:** Write a function that counts the occurrences of each element in an array and returns an object `{ element: count }`.
8. **Find and Replace a Value in an Array:** Given an array and a target value, replace all occurrences of that value with a new value.
9. **Use Switch Case with Arrays:** Create a menu-driven program where a user selects an option to sort, reverse, or find an element in an array.

10. **Convert an Array of Objects to a Single Object:** Given an array of objects where each object has a key-value pair, convert it into a single object.
11. **Rotate an Array K Times to the Right:** Given an array and a number K , rotate it to the right K times.
12. **Extract Even-Indexed Elements from an Array:** Write a function that extracts and returns elements at even indices from an array.
13. **Find the Median of an Array:** Write a function that finds the median value of an array of numbers.
14. **Count Odd and Even Numbers in an Array:** Given an array of numbers, return an object with counts of even and odd numbers.
15. **Use `.some()` and `.every()` Methods:** Write a function that checks if all numbers in an array are positive and if at least one number is negative.
16. **Flatten a Nested Array:** Convert a multi-dimensional array into a single-dimensional array.
17. **Remove Falsy Values from an Array:** Remove all `false`, `null`, `0`, `""`, `undefined`, and `NaN` values from an array.
18. **Convert an Array to a CSV String:** Write a function that converts an array into a comma-separated string.
19. **Reverse a 2D Array (Matrix Transposition):** Given a 2D array (matrix), swap rows and columns.
20. **Create an Array of Unique Random Numbers:** Write a function that generates an array of unique random numbers within a given range.
21. **Find the Intersection of Multiple Arrays:** Given multiple arrays, return an array containing only the common elements among all arrays.
22. **Find the Longest Increasing Subsequence in an Array:** Write a function to find the longest consecutive increasing numbers in an array.
23. **Find the Difference Between Maximum and Minimum in an Array:** Given an array of numbers, return the difference between the highest and lowest values.
24. **Sort an Array of Objects by a Property:** Given an array of objects (students with marks), sort them in descending order based on marks.
25. **Convert an Array of Objects into an Object:** Given an array of objects with `id` as a key, convert it into a single object where each key is an `id` and its value is the object itself.
26. **Split an Array into Smaller Chunks:** Write a function that splits an array into chunks of a given size.
27. **Find Duplicate Values in an Array:** Given an array, return a new array containing only the duplicate values.
28. **Implement a Custom `map()` Function:** Write your own function that works like the `.map()` method for arrays.

29. **Filter an Array Based on Conditions:** Given an array of objects (employees with age and salary), filter out employees earning more than 50000 and younger than 30.
30. **Create a Dynamic Menu Using Switch and Arrays:** Implement a menu-driven program where the user can add, remove, sort, and display an array of numbers using a `switch` statement.
-

7. JavaScript Map and Sets questions

Basic Level (Maps + Sets + Variables + Loops + Operators)

1. **Create a Map and Add Key-Value Pairs:** Create a `Map` and add at least 5 key-value pairs to it.
2. **Retrieve a Value from a Map:** Given a `Map`, retrieve the value associated with a given key.
3. **Check If a Key Exists in a Map:** Write a function that checks if a specific key exists in a `Map`.
4. **Iterate Over a Map:** Use a `forEach` loop to iterate over a `Map` and print each key-value pair.
5. **Delete a Key from a Map:** Write a function that removes a given key from a `Map`.
6. **Find the Size of a Map:** Create a `Map` and determine its size using the `.size` property.
7. **Convert a Map to an Array:** Write a function that converts a `Map` into an array of key-value pairs.
8. **Create a Set and Add Unique Values:** Create a `Set` and add at least 5 unique values to it.
9. **Check If a Value Exists in a Set:** Write a function that checks if a specific value exists in a `Set`.
10. **Delete an Element from a Set:** Write a function that removes a specific value from a `Set`.
11. **Find the Size of a Set:** Create a `Set` and determine its size using the `.size` property.
12. **Iterate Over a Set:** Use a `forEach` loop to iterate over a `Set` and print each element.
13. **Convert a Set to an Array:** Write a function that converts a `Set` into an array.
14. **Find Unique Elements in Two Arrays Using Sets:** Given two arrays, use a `Set` to return unique elements from both.
15. **Convert an Array to a Set and Back to an Array:** Write a function that removes duplicates from an array by converting it to a `Set`, then converting it back to an array.
16. **Check If Two Sets Are Equal:** Write a function that checks whether two `Sets` have the same elements.
17. **Find the First Repeating Element in an Array Using a Set:** Given an array, find the first repeating element using a `Set`.

18. **Find the First Non-Repeating Element in an Array Using a Map:** Given an array, find the first non-repeating element using a `Map`.
 19. **Sort a Map by Its Values:** Given a `Map` of names and ages, sort it by age in ascending order.
 20. **Merge Two Maps Into One:** Write a function that merges two `Maps`, giving preference to the second map in case of duplicate keys.
-

Intermediate Level (Maps + Sets + Arrays + Objects + Loops + Switch + Conditions)

1. **Find the Intersection of Two Sets:** Write a function that takes two `Sets` and returns a new `Set` containing only the common elements.
2. **Find the Union of Two Sets:** Write a function that takes two `Sets` and returns a new `Set` containing all elements from both.
3. **Find the Difference Between Two Sets:** Write a function that returns the elements that are in the first `Set` but not in the second.
4. **Count the Occurrences of Words in an Array Using a Map:** Given an array of words, create a `Map` that counts how many times each word appears.
5. **Convert an Object to a Map:** Write a function that converts a JavaScript object into a `Map`.
6. **Convert a Map to an Object:** Write a function that converts a `Map` back into an object.
7. **Remove Duplicates from an Array Using a Set:** Given an array with duplicate values, return a new array with only unique values using a `Set`.
8. **Use a Map to Store Student Grades:** Create a `Map` where student names are keys and their grades are values. Implement a function to retrieve a student's grade.
9. **Filter a Map Based on Values:** Given a `Map` of employees and their salaries, return a new `Map` containing only employees who earn more than 50000.
10. **Implement a Menu-Driven Program Using a Switch Statement and Maps:** Create a program that allows users to add, remove, update, and display key-value pairs in a `Map` using a `switch` statement.
11. **Remove All Keys from a Map That Have a Specific Value:** Given a `Map`, remove all key-value pairs where the value is `null` or `undefined`.
12. **Find Common Keys in Two Maps:** Given two `Maps`, return a new `Set` containing only the keys that exist in both.
13. **Group Words by Their Length Using a Map:** Given an array of words, create a `Map` where the keys are word lengths and the values are arrays of words of that length.

14. **Use a Map to Store Product Prices and Find the Most Expensive Product:** Create a `Map` where keys are product names and values are their prices, then find the most expensive product.
 15. **Check If a Set is a Subset of Another Set:** Write a function that determines if one `Set` is completely contained within another.
 16. **Find the Difference of Two Sets:** Write a function that returns elements present in the first `Set` but not in the second.
 17. **Convert a String into a Character Frequency Map:** Given a string, return a `Map` where keys are characters and values are their occurrences.
 18. **Find the Intersection of Three Sets:** Given three `Sets`, find the common elements among all of them.
 19. **Use a Set to Keep Track of Unique Users in a Web Application:** Simulate a system where unique user IDs are stored in a `Set`, preventing duplicates.
 20. **Implement a Caching System Using a Map:** Create a simple caching system where a `Map` stores recently used values and removes old values after a limit is reached.
-

8. JavaScript Strings, String Methods, Numbers, and Random questions

Basic Level (Strings + Numbers + Random + Operators + Loops)

1. **Reverse a String:** Write a function that takes a string and returns the reversed string.
2. **Check if a String is a Palindrome:** Write a function that checks whether a given string is a palindrome (ignoring case and spaces).
3. **Convert a String to Uppercase and Lowercase:** Given a string, return both its uppercase and lowercase versions.
4. **Count the Number of Vowels in a String:** Write a function that counts the vowels (a, e, i, o, u) in a given string.
5. **Extract the First N Characters of a String:** Given a string and a number N, return the first N characters of the string.
6. **Find the Length of a String Without Using .length:** Implement a function that counts characters in a string without using `.length`.
7. **Generate a Random Number Between 1 and 100:** Write a function that generates a random number between 1 and 100.

8. **Generate a Random Floating-Point Number Between Two Values:** Write a function that takes two numbers as input and returns a random floating-point number between them.
 9. **Round a Number to 2 Decimal Places:** Given a floating-point number, round it to two decimal places.
 10. **Convert a Number to a String and Vice Versa:** Given a number, convert it to a string and back to a number.
 11. **Convert the First Letter of Each Word in a String to Uppercase:** Given a string, capitalize the first letter of each word.
 12. **Reverse Words in a Sentence:** Given a sentence, reverse the order of words while keeping the letters in each word intact.
 13. **Check If a String is an Anagram:** Write a function to check if two given strings are anagrams of each other.
 14. **Find the Longest Word in a Sentence:** Write a function that returns the longest word in a given sentence.
 15. **Check If a Number is Prime:** Given a number, write a function to check if it's a prime number.
 16. **Find the Factorial of a Number:** Write a function that takes a number as input and returns its factorial.
 17. **Generate a Random Alphanumeric String:** Write a function that generates a random alphanumeric string of a given length.
 18. **Round a Number to the Nearest Integer:** Given a floating-point number, round it to the nearest whole number.
 19. **Check If a Number is Even or Odd Using a String:** Convert the number to a string and determine if it's even or odd.
 20. **Count the Occurrences of Each Letter in a String:** Given a string, return an object where keys are letters and values are their counts.
-

Intermediate Level (Strings + Numbers + Random + Arrays + Objects + Loops + Conditions + Switch + Maps + Sets)

1. **Replace All Occurrences of a Word in a String:** Given a string and a word, replace all occurrences of that word with another word.
2. **Find the Most Frequent Character in a String:** Given a string, find the character that appears most frequently.
3. **Shuffle the Characters of a String Randomly:** Write a function that shuffles the characters of a string randomly.
4. **Extract Digits from a String:** Given a string containing letters and numbers, extract only the numeric digits and return them as a number.
5. **Check If a String Contains Only Numbers:** Write a function that returns `true` if a string contains only numeric characters.

6. **Generate a Random Password:** Write a function that generates a random password of a given length, containing uppercase, lowercase, numbers, and special characters.
 7. **Sort Words in a String Alphabetically:** Given a sentence, return the words sorted in alphabetical order.
 8. **Find the Sum of All Digits in a String:** Given a string containing numbers, find the sum of all numeric digits.
 9. **Simulate a Dice Roll Using `Math.random()`:** Create a function that simulates rolling a 6-sided dice and returns the result.
 10. **Create a String-Based Calculator:** Write a function that takes a string containing a mathematical expression (e.g., "10 + 5 * 2") and evaluates the result.
 11. **Find the Smallest and Largest Number in a String:** Given a string containing numbers, return the smallest and largest numbers found.
 12. **Extract All Words That Start with a Specific Letter:** Given a sentence and a letter, return all words that start with that letter.
 13. **Find the ASCII Code of Each Character in a String:** Write a function that returns an array of ASCII codes for each character in a string.
 14. **Generate a Random Color Code (Hex or RGB):** Write a function that generates a random color in hex (`#RRGGBB`) or RGB format.
 15. **Generate a Random OTP of Given Length:** Create a function that generates a random numeric OTP of a specified length.
 16. **Format a Number with Commas as Thousands Separators:** Given a number, format it using commas (e.g., 1000000 → "1,000,000").
 17. **Check If a Sentence is a Pangram:** Write a function to check if a sentence contains every letter of the alphabet at least once.
 18. **Replace All Vowels in a String with a Random Symbol:** Given a string, replace each vowel with a randomly chosen symbol from @, #, \$, %, &.
 19. **Simulate a Coin Toss Using `Math.random()`:** Write a function that randomly returns "Heads" or "Tails".
 20. **Create a Dice Rolling Game Between Two Players:** Simulate a game where two players roll a dice (1–6) using `Math.random()` and determine the winner.
-

9. JavaScript Dates and Math Methods questions

Basic Level (Dates + Math Methods + Operators + Loops)

1. **Get the Current Date and Time:** Write a function that returns the current date and time in `YYYY-MM-DD HH:MM:SS` format.
2. **Find the Difference Between Two Dates in Days:** Given two dates, return the difference between them in days.
3. **Check If a Year is a Leap Year:** Write a function that checks if a given year is a leap year or not.
4. **Find the Day of the Week for Any Date:** Given a date string (`YYYY-MM-DD`), return the name of the weekday (e.g., "Monday").
5. **Round a Decimal Number to the Nearest Integer Using `Math.round()`:** Given a decimal number, round it using `Math.round()`.
6. **Find the Square Root of a Number Using `Math.sqrt()`:** Given a number, return its square root using `Math.sqrt()`.
7. **Get the Absolute Value of a Number Using `Math.abs()`:** Write a function that returns the absolute difference between two numbers.
8. **Find the Smallest and Largest Numbers in an Array Using `Math.min()` and `Math.max()`:** Given an array of numbers, find the smallest and largest values.
9. **Generate a Random Number Between Two Values and Round It:** Write a function that generates a random number between `min` and `max`, then rounds it to the nearest whole number.
10. **Find the Ceiling and Floor of a Decimal Number:** Given a decimal number, return both its ceiling (`Math.ceil()`) and floor (`Math.floor()`).

Intermediate Level (Dates + Math Methods + Arrays + Objects + Loops + Conditions + Switch + Maps + Sets)

1. **Calculate Age Based on Date of Birth:** Given a birthdate (`YYYY-MM-DD`), return the person's age in years.
2. **Find the Number of Days Left Until the Next New Year:** Write a function that returns the number of days remaining until the next January 1st.
3. **Find the First Monday After a Given Date:** Given a date, return the first Monday that occurs after that date.
4. **Convert a Timestamp to a Readable Date:** Given a timestamp (e.g., `1700000000000`), convert it to a human-readable date format.
5. **Calculate the Power of a Number Using `Math.pow()`:** Write a function that raises a number to a given power using `Math.pow()`.
6. **Simulate a Lottery System Using Random and Dates:** Generate a 6-digit lottery number using `Math.random()` and timestamp it with the current date.

7. **Find the First Day of the Current Month:** Write a function that returns the first day of the current month in YYYY-MM-DD format.
 8. **Find the Last Day of Any Month and Year:** Given a month and year, return the last date of that month.
 9. **Generate a Future Date After N Days:** Given a starting date and a number N , return the date that occurs N days later.
 10. **Create a Countdown Timer for an Event:** Given a future date, calculate the number of days, hours, minutes, and seconds left until that date.
-

10. JavaScript questions focusing on Functions (Regular & Arrow Functions)

Basic Level (Function Basics)

1. **Write a function** that takes two numbers as arguments and returns their sum.
2. **Convert the function** from question 1 into an arrow function.
3. **Write a function** that takes a string as input and returns its length.
4. **Create a function** that checks if a number is even or odd.
5. **Write an arrow function** that returns the square of a number.
6. **Write a function** that returns the absolute difference between two numbers.
7. **Create an arrow function** that returns the last character of a string.
8. **Write a function** that takes a number and returns "Positive", "Negative", or "Zero".
9. **Define a function** that prints "Hello, World!" when called.
10. **Write a function** that swaps two variables without using a third variable.
11. **Write a function** that takes two numbers and returns the smaller one.
12. **Create an arrow function** that concatenates two strings with a space in between.
13. **Write a function** that takes a string and returns "Yes" if it contains "JavaScript", otherwise returns "No".
14. **Create a function** that converts Celsius to Fahrenheit.
15. **Write a function** that checks whether a number is prime.

Intermediate Level (Function Expressions & Callbacks)

1. **Write a function expression** (not a function declaration) that returns the cube of a number.
2. **Create a function** that accepts another function as an argument (callback) and executes it.
3. **Write an arrow function** that reverses a given string.
4. **Create a function** that returns the larger of two numbers using a ternary operator.
5. **Write a function** that returns "Hello, [name]!" when given a name as an argument.
6. **Create a function expression** that takes a string and returns it in uppercase.
7. **Write a function** that accepts a function as an argument and calls it three times.
8. **Create an arrow function** that returns the number of vowels in a given string.
9. **Write a function** that checks if an array contains a given element.
10. **Create a function** that takes a number and returns "Fizz" if divisible by 3, "Buzz" if divisible by 5, and "FizzBuzz" if divisible by both.
11. **Create a function expression** that returns the length of the longest word in a sentence.
12. **Write a function** that accepts another function and an argument, then calls the function with that argument.
13. **Create an arrow function** that takes an array of numbers and returns the sum of all elements.
14. **Write a function** that takes a number `n` and returns an array of its factors.
15. **Create a function** that generates a random hexadecimal color code (e.g., #A3F9B2).

Advanced Level (Higher-Order Functions & Closures)

1. **Write a higher-order function** that takes an array and a function, then applies the function to each element of the array.
2. **Create a function** that returns another function to multiply a number by a given factor (closure).
3. **Write an IIFE** (Immediately Invoked Function Expression) that logs "IIFE executed" to the console.
4. **Create a recursive function** to calculate the factorial of a number.
5. **Write a function** that accepts an array of numbers and returns a new array where each number is doubled using `.map()`.
6. **Write a function that returns another function**, which when called, returns "JavaScript is awesome!".
7. **Create a function** that logs "Start" → waits 2 seconds → logs "End" using `setTimeout()`.
8. **Write an IIFE** that immediately logs "Executed immediately" to the console.

9. **Create a function** that generates a random number between `min` and `max` values (inclusive).
10. **Write a recursive function** to calculate the sum of digits in a number.
11. **Write a function that returns another function**, which adds a given number to its argument.
12. **Create a function** that executes a given function only once (implementing `once` behavior).
13. **Write an IIFE** that counts the number of times a button is clicked (using closures).
14. **Create a function** that accepts an array of numbers and returns an array with only the unique values.
15. **Write a recursive function** to calculate the Greatest Common Divisor (GCD) of two numbers.

Expert Level (Advanced Function Concepts)

1. **Write a function using** `.filter()` that returns only the even numbers from an array.
 2. **Create a function using** `.reduce()` to find the sum of an array of numbers.
 3. **Write a function** that memoizes another function's result to optimize performance.
 4. **Create a function** that uses `.forEach()` to log each element of an array with its index.
 5. **Write a function using** `.sort()` that sorts an array of numbers in descending order.
 6. **Write a function using** `.map()` that converts an array of names to uppercase.
 7. **Create a function using** `.reduce()` that finds the product of all elements in an array.
 8. **Write a function** that calculates the sum of even numbers in an array using `.filter()` and `.reduce()`.
 9. **Implement a function** that delays execution of another function by `n` milliseconds (debouncing).
 10. **Write a function using** `.sort()` that sorts an array of objects by a property "age".
 11. **Write a function using** `.map()` that returns an array of string lengths from an array of strings.
 12. **Create a function using** `.reduce()` that finds the longest string in an array.
 13. **Write a function** that returns a new function which logs "Function executed" only once.
 14. **Implement a function** that throttles another function to be executed at most once per second.
 15. **Write a function using** `.sort()` that sorts an array of objects by their "name" property in alphabetical order.
-

11. JavaScript questions focusing on Asynchronous JavaScript (Callbacks, Promises, Async/Await)

Basic Level (Understanding Callbacks & Promises)

1. **Write a function** that takes a callback and executes it after logging "Start" and before logging "End".
2. **Create a function** that simulates a delay using `setTimeout()` and logs "Hello, after 2 seconds" after 2 seconds.
3. **Write a function** that uses `setInterval()` to log "Running..." every second.
4. **Convert a function** using callbacks into a function that returns a Promise instead.
5. **Create a simple Promise** that resolves with "Promise Resolved" after 3 seconds and then logs the result.
6. **Write a function** that takes a callback and calls it after a delay of 3 seconds.
7. **Create a function** that logs "Tick" every second using `setInterval()` but stops after 5 times.
8. **Write a function** that simulates an API call using `setTimeout()` and returns "Data fetched" after 2 seconds.
9. **Implement a function** that takes two callbacks: one for success and one for failure, and randomly calls one of them.
10. **Convert a callback-based function** into a Promise-based function.

Intermediate Level (Handling Promises & Async/Await)

1. **Write a function** that returns a Promise which resolves if a number is even and rejects if it's odd.
2. **Create a Promise chain** that fetches a user's name, then their age, and then logs "User: [name], Age: [age]".
3. **Write an async function** that waits for a Promise to resolve and then logs the result.
4. **Use `Promise.all()`** to wait for multiple Promises and log all resolved values.
5. **Write an async function** that fetches data from an API using `fetch()` and logs the response.
6. **Create a Promise** that resolves with "Success" if a random number is greater than 0.5 and rejects with "Failure" otherwise.

7. **Write a function** that returns a Promise that resolves after a given number of milliseconds.
8. **Implement a function** that chains multiple `.then()` calls to transform a value step by step.
9. **Write an async function** that waits for a Promise and then returns "Task Complete".
10. **Use `Promise.all()`** to wait for three Promises and return an array of their results.

Advanced Level (Handling Errors & Optimizing Async Code)

1. **Write a function** that simulates a failed API call (using `setTimeout`) and handles the error using `.catch()`.
2. **Create an async function** that retries an API call up to 3 times before failing.
3. **Write a function using `Promise.race()`** that resolves the fastest of three Promises.
4. **Convert a callback-based function** (like `fs.readFile`) into a Promise-based function using `util.promisify()`.
5. **Write an async function** that makes multiple API calls and logs them one by one using `for await...of`.
6. **Create a function** that simulates an API request but fails 50% of the time, and handle errors using `.catch()`.
7. **Write an async function** that handles multiple API calls and ensures that all of them succeed before logging "All data loaded".
8. **Implement a function** that logs "Processing..." while waiting for an async function to complete.
9. **Write a function using `Promise.race()`** that resolves when the fastest Promise completes.
10. **Implement a retry mechanism** where a failing Promise is retried up to 3 times before throwing an error.

Expert Level (Deep Dive into Async JavaScript)

1. **Implement a debounce function** that delays executing a callback until after a specified delay.
2. **Write a throttle function** that ensures a function executes at most once every `n` milliseconds.
3. **Use `Promise.allSettled()`** to handle multiple Promises that might fail and log their status.
4. **Create an event loop simulation** that demonstrates how JavaScript handles asynchronous tasks using `setTimeout`, `Promise.resolve()`, and normal synchronous code.
5. **Write a function** that measures the time taken for an async function to execute and logs the time taken.
6. **Write a function** that logs "Microtask" inside a Promise and "Macrotask" inside `setTimeout()` and observe the execution order.
7. **Create a debounce function** that delays executing a function until after a specified amount of time.

8. **Implement a throttle function** that ensures a function executes at most once every 2 seconds.
 9. **Write an async function** that measures the execution time of another async function.
 10. **Demonstrate the difference** between `Promise.all()`, `Promise.any()`, `Promise.race()`, and `Promise.allSettled()` using examples.
-

12. JavaScript questions focusing on Error Handling & Debugging

Basic Level (Understanding Errors & Try-Catch)

1. **Write a function** that tries to parse a JSON string and catches errors if the string is invalid.
 2. **Create a function** that divides two numbers but throws an error if the denominator is zero.
 3. **Write a try-catch block** that handles an error when accessing a property of `undefined`.
 4. **Use `console.log()`, `console.warn()`, and `console.error()`** to debug a function that performs addition.
 5. **Write a function** that safely accesses a deeply nested object property and handles the error if it doesn't exist.
 6. **Write a function** that reads a user's input and throws an error if the input is empty.
 7. **Use try-catch** to handle an error inside a function that converts a string to a number using `parseInt()`.
 8. **Write a function** that catches an error when accessing an invalid array index.
 9. **Use `console.assert()`** to check if a number is greater than 10 and log an error if it's not.
 10. **Create a function** that throws an error if an input is not a valid email address.
-

Intermediate Level (Custom Errors & Debugging Techniques)

1. **Create a custom error class** called `InvalidAgeError` that is thrown when age is negative.
2. **Write a function** that uses `finally{}` to log "Execution Completed" whether an error occurs or not.
3. **Use `console.table()`** to display an array of objects in a structured format for debugging.
4. **Write a function** that logs all function calls, arguments, and return values using `console.trace()`.
5. **Use `debugger`; in the browser console** to pause execution and inspect variables inside a loop.
6. **Write a function** that logs function execution details (arguments, return value, errors) using `console.log()`.

7. **Create a try-catch block** that throws a custom error message "Invalid Operation" when performing an illegal operation.
 8. **Write a function** that uses `console.group()` and `console.groupEnd()` to structure logs inside a loop.
 9. **Use debugger;** inside a recursive function to analyze its execution.
 10. **Write an async function** that handles API errors gracefully and logs "API request failed" if an error occurs.
-

Advanced Level (Handling Asynchronous Errors & Stack Tracing)

1. **Write an async function** that catches and logs errors when making a failing API call using `fetch()`.
 2. **Create a function** that wraps another function and logs errors instead of letting them crash the program.
 3. **Write a function** that implements global error handling using `window.onerror` (browser) or `process.on('uncaughtException')` (Node.js).
 4. **Use try-catch inside an async/await function** to handle Promise rejections properly.
 5. **Write a function** that generates an error with a custom error message and stack trace using `Error().stack`.
 6. **Implement a function** that logs errors to a file (Node.js) instead of printing them to the console.
 7. **Write an async function** that waits for multiple API calls using `Promise.all()`, and logs errors for any rejected promises.
 8. **Use try-catch-finally** to handle errors and always execute "Cleanup completed" at the end.
 9. **Write a function** that uses `Object.defineProperty()` to create an error-safe read-only object.
 10. **Create a function** that uses `setTimeout()` inside try-catch and explain why the error is not caught.
-

Expert Level (Optimizing Debugging & Performance Monitoring)

1. **Implement a higher-order function** that wraps another function and logs how long it takes to execute.
2. **Use console.time() and console.timeEnd()** to measure the execution time of a loop running 100,000 times.
3. **Write a function** that logs memory usage before and after executing a function in Node.js.
4. **Use performance.now()** to measure the time taken for an API request to complete.
5. **Write a function** that detects memory leaks by tracking variable references inside a closure.
6. **Write a function** that creates an infinite loop and logs "Loop interrupted" when detected using an error.

7. **Use `console.trace()`** inside a function that is deeply nested in multiple calls and analyze the call stack.
 8. **Write an error logger** that captures both `window.onerror` (browser) and `process.on('uncaughtException')` (Node.js).
 9. **Create a performance profiler** that measures how long a function takes using `performance.mark()` and `performance.measure()`.
 10. **Write a function** that wraps another function in a try-catch block and automatically retries execution if an error occurs.
-

13. JavaScript questions focusing on Object-Oriented Programming (OOP)

Basic Level (Classes, Objects, Constructor Functions)

1. **Create a class** named `Person` with properties `name` and `age`, and instantiate an object from it.
2. **Write a class** `Car` with properties `brand` and `model`, and a method `start()` that logs "Car started".
3. **Create a constructor function** for `Student` that initializes `name` and `grade`, and add a method to display details.
4. **Write a class** `Animal` that has a constructor with a `species` property and a method `speak()` that logs "Animal makes a sound".
5. **Create multiple instances** of the `Person` class and log their details using a method inside the class.
6. **Create a class** `Book` with properties `title`, `author`, and a method `getDetails()` that returns a formatted string.
7. **Write a class** `Movie` with a constructor that takes `title` and `director`, and create an instance of it.
8. **Add a method** to a `Person` class that returns the person's full name by combining `firstName` and `lastName`.
9. **Create a class** `Laptop` with a method `turnOn()` that logs "Laptop is now on."
10. **Write a function** that dynamically adds a new method `greet()` to an existing `User` class.
11. **Create a class** `Phone` with properties `brand` and `model`, and a method `call()` that logs "Calling...".
12. **Write a class** `Laptop` with properties `brand`, `processor`, and `RAM`, and a method `getSpecs()` that returns a string of its specs.
13. **Create a class** `TV` with a method `increaseVolume()` that increases the volume by 1.

14. **Write a class** `Account` with a private `#balance` property and a method to deposit money into the account.
 15. **Create an object** using an object literal `{}` and then add a method dynamically using `Object.defineProperty()`.
-

Intermediate Level (Inheritance, Encapsulation, Getters & Setters)

1. **Extend the `Animal` class** to create a `Dog` class that overrides the `speak()` method to log "Bark!".
 2. **Use getters and setters** to create a `BankAccount` class that sets a balance and ensures it cannot be negative.
 3. **Create a class** `Rectangle` with properties `width` and `height`, and a method `area()` to return the area.
 4. **Implement encapsulation** by making properties of a `User` class private using `#privateVariable`.
 5. **Write a class** `Circle` that has a private property `#radius`, and provide a method to get and set its value safely.
 6. **Create a parent class** `Employee` and a child class `Manager` that extends `Employee`.
 7. **Use getters and setters** to create a `Temperature` class that converts Celsius to Fahrenheit.
 8. **Make properties private** in a `BankAccount` class using `#privateBalance` and allow deposits/withdrawals through methods.
 9. **Create a `Vehicle` class** and extend it with `Car` and `Bike` classes, overriding a method `honk()`.
 10. **Write a class** `Student` where `name` and `rollNumber` are private, and only getters allow access.
 11. **Write a `Person` class** with private properties `#name` and `#age` and create getter and setter methods.
 12. **Create a prototype method** for an existing class `Array` to add a method `first()` that returns the first element.
 13. **Extend the `Vehicle` class** to create `Car` and `Bike` classes and override a method `drive()`.
 14. **Use `Object.create()`** to create an object with a specific prototype and access its methods.
 15. **Implement method overriding** in a class `Animal`, with subclasses `Dog` and `Cat` that override `speak()`.
-

Advanced Level (Polymorphism, Static Methods, Prototypes)

1. **Implement polymorphism** by creating a `Shape` class with a method `draw()`, and then override it in `Circle` and `Square` classes.
2. **Create a `MathUtility` class** with a static method `square(num)` that returns the square of a number.
3. **Use `Object.create()`** to create an object with a prototype and access a method from the prototype.
4. **Write a prototype-based inheritance example** where `Employee` extends from `Person`.

5. Use **Object.assign()** to copy properties from one object to another and log the final object.
6. Demonstrate **polymorphism** by creating a `Payment` class with a method `pay()`, and override it in `CreditCardPayment` and `PayPalPayment`.
7. Write a **Logger class** with a static method `logMessage()` that logs messages with a timestamp.
8. Modify the **prototype** of an existing object to add a method `capitalize()` to all string objects.
9. Override the built-in `toString()` method in a `Product` class to return a custom string representation.
10. Create an **array prototype method** that extends the functionality of JavaScript's built-in array to add a method `sum()`.
11. Write a class `MathOperations` with a static method `add(x, y)` that returns the sum of two numbers.
12. Implement **abstraction** by creating a base class `Shape` with an abstract method `calculateArea()` that must be implemented in subclasses.
13. Create a **mixin** to add logging functionality to multiple classes (`User`, `Admin`, `Moderator`).
14. Use the **Factory Pattern** to create different types of `Animals` (`Dog`, `Cat`, `Bird`) dynamically.
15. Create a **singleton class** `ConfigManager` that stores configuration settings and ensures only one instance is created.

Expert Level (Mixins, Factory Functions, Method Overriding)

1. Create a **mixin** that adds a `fly()` method to multiple classes like `Bird` and `Airplane`.
2. Write a **factory function** that returns a new `Vehicle` object with properties based on parameters passed.
3. Create a **Singleton class** that ensures only one instance of a `Database` object is created.
4. Override the `toString()` method in a `Product` class to return a formatted string of its details.
5. Demonstrate **method chaining** in a class by returning `this` from multiple methods like `.setName()`, `.setAge()`.
6. Create a **mixin** that adds a `log()` method to different classes (`User`, `Admin`) without modifying their structure.
7. Use a **factory function** to create different types of `Animal` objects based on input parameters.
8. Implement the **Singleton pattern** in a `Database` class to ensure only one instance exists.
9. Create a class `car` where multiple methods (`setBrand()`, `setModel()`, `setYear()`) return `this` for method chaining.

10. **Write a function** that dynamically adds a method to an existing class without modifying the original source code.
 11. **Demonstrate composition** by creating a `Car` class that has an `Engine` object instead of extending from `Vehicle`.
 12. **Use a Proxy object** to validate changes to an object's properties before allowing modifications.
 13. **Create a decorator function** that adds caching functionality to any function passed to it.
 14. **Implement the Observer Pattern** where a `NewsChannel` class notifies multiple subscribers whenever news is updated.
 15. **Write a performance-optimized function** that creates thousands of objects efficiently using prototypes instead of classes.
-

14. JavaScript questions focusing on Event Loop & Performance Optimization

Basic Level (Event Loop, Call Stack, Macrotasks & Microtasks)

1. **Explain the Event Loop** and demonstrate it using `setTimeout()` and `console.log()`.
 2. **What is the Call Stack?** Write a simple recursive function and observe the stack behavior.
 3. **Create a JavaScript program** that uses `setTimeout()`, `Promise.resolve()`, and `console.log()` and explain the execution order.
 4. **Demonstrate the difference** between a Microtask (`Promise.then()`) and a Macrotask (`setTimeout()`).
 5. **Write a function** that uses `queueMicrotask()` to schedule a microtask.
 6. **Explain the order of execution** for `console.log()`, `setTimeout()`, `Promise.resolve()`, and `setImmediate()`.
 7. **What happens when you use `setTimeout(fn, 0)`?** Show an example where it doesn't execute immediately.
 8. **Write a function** that logs "Start", delays execution using `setTimeout()`, and logs "End". Explain the output.
 9. **What is the role of Web APIs** in the Event Loop? Demonstrate with an example.
 10. **Demonstrate how microtasks execute before macrotasks** using `Promise.then()` and `setTimeout()`.
-

Intermediate Level (Web APIs, setTimeout vs setImmediate, Performance Bottlenecks)

1. **Create a program** where `setTimeout(fn, 0)` executes after a long-running synchronous task.
 2. **What happens when you call** `setTimeout()` **inside a for loop?** Demonstrate with an example.
 3. **Write a function** that delays execution using `requestAnimationFrame()` **instead of** `setTimeout()`.
 4. **Simulate blocking the Event Loop** by writing an infinite `while` loop inside a function and observe browser behavior.
 5. **Write an optimized function** that batches multiple DOM updates using `requestAnimationFrame()`.
 6. **Write a function** that logs numbers from 1 to 5, using both `setTimeout` and `Promise`. Explain the order of execution.
 7. **Optimize a function** that updates the DOM multiple times by using `requestAnimationFrame()`.
 8. **Implement a debounce function** that waits 300ms after the last keypress before executing.
 9. **Implement a throttle function** that limits execution to once every second.
 10. **Compare the performance of a for loop, forEach(), and map()** when processing a large dataset.
-

Advanced Level (Debouncing, Throttling, Memory Leaks, Garbage Collection)

1. **Implement a debounce function** that limits the rate of a function execution.
 2. **Implement a throttle function** that allows execution once every X milliseconds.
 3. **Demonstrate memory leaks** using global variables and closures, then fix them.
 4. **Write a function** that optimizes event listeners using `passive: true`.
 5. **Analyze performance bottlenecks** using `performance.now()` to measure execution time.
 6. **Create a Web Worker** to perform a heavy computation without blocking the main thread.
 7. **Demonstrate requestIdleCallback()** to execute a background task when the browser is idle.
 8. **Analyze a function's execution time** using `performance.now()`.
 9. **Explain and optimize a slow recursive function** using Tail Call Optimization (TCO).
 10. **Write a function that batches API requests** instead of sending them individually.
-

Expert Level (Web Workers, Async Optimization, Task Prioritization)

1. **Use a Web Worker** to run a CPU-intensive calculation without blocking the main thread.

2. **Demonstrate `requestIdleCallback()`** by scheduling a low-priority task when the browser is idle.
 3. **Write an optimized data-fetching function** that uses caching to avoid redundant network requests.
 4. **Optimize a slow function** using Memoization and measure the difference.
 5. **Explain and implement** async Task Prioritization using `setImmediate()` or `MessageChannel()`.
 6. **Demonstrate how `MessageChannel` can be used** to prioritize async tasks over `setTimeout()`.
 7. **Write an optimized search function** that caches previous results to avoid redundant calculations.
 8. **Simulate a memory leak** using closures and global variables, then fix it.
 9. **Optimize event listeners** by using `once: true` and `passive: true`.
 10. **Use Web Performance APIs** to measure and optimize the load time of a web page.
-

15. JavaScript questions focusing on DOM Manipulation, covering Selectors, Events, Attributes, Styles, Traversal, and Dynamic Updates. ☐

Basic Level (Selecting and Modifying Elements)

1. **Write a JavaScript function** to select an element by ID and change its text content.
2. **Select all `<p>` elements** on a page and change their font color to blue using JavaScript.
3. **Create a button** that, when clicked, hides an element using `display: none`.
4. **Write a script** that adds a new `<li>` item to an existing `<ul>` list when a button is clicked.
5. **Use `querySelectorAll()`** to select all elements with class `"box"` and change their background color.
6. **Write a JavaScript function** that changes the text of all `<h2>` elements on the page.
7. **Select the first and last child elements** of a `<ul>` list and apply different colors to them.
8. **Create a button** that, when clicked, changes the background color of the entire page.
9. **Modify the inner HTML** of a `<div>` to include a new `<span>` element dynamically.

10. **Write a function** that adds a border to all images when hovered over.
 11. **Write JavaScript code** to create a `<p>` element and append it to a `<div>` with the ID "container".
 12. **Change the color of all even-indexed `<li>` elements** in an unordered list.
 13. **Write a function** that updates the text of a button when clicked (e.g., "Click Me" → "Clicked!").
 14. **Modify an image's `alt` attribute** using JavaScript.
 15. **Create a function** that adds a CSS class "highlight" to a paragraph when the mouse is over it.
-

Intermediate Level (Events, Attributes, and Traversing the DOM)

1. **Attach a click event** to a button that toggles the visibility of a paragraph.
 2. **Write a function** that adds an "active" class to a clicked element and removes it from others.
 3. **Modify an image's `src` attribute** dynamically when clicking a button.
 4. **Use `nextElementSibling` and `previousElementSibling`** to navigate between elements in a list.
 5. **Retrieve and change the value** of an `<input>` field when a button is clicked.
 6. **Create an input field and a button**—when clicked, display the input value in a `<p>` tag.
 7. **Use `dataset` attributes** to store extra information inside an element and retrieve it using JavaScript.
 8. **Write a script** that changes an element's style every time the user clicks a button.
 9. **Prevent the default form submission behavior** and display the form data instead.
 10. **Write a script** that dynamically adds an item to a dropdown menu on button click.
 11. **Create an input field** that updates a `<span>` with live text as the user types.
 12. **Build a character counter** for a textarea that updates the count dynamically.
 13. **Write a JavaScript function** that disables a button after it's clicked once.
 14. **Write a function** to show a confirmation box when a user clicks the "Delete" button.
 15. **Use `innerHTML` to insert a list of items** dynamically from an array.
-

Advanced Level (Creating and Removing Elements Dynamically)

1. **Write JavaScript code** to create a new `<div>` element with text and append it to the document.
 2. **Write a function** that removes the last item from a list dynamically.
 3. **Use `classList.add()` and `classList.remove()`** to toggle CSS classes on a div when clicked.
 4. **Write a script** that adds multiple list items to a `<ul>` dynamically using `document.createElement()`.
 5. **Use `insertAdjacentHTML()`** to insert a new element before an existing paragraph.
 6. **Implement a light/dark mode toggle** that switches styles when a button is clicked.
 7. **Write an event listener** that only triggers on `<li>` items inside a `<ul>` using event delegation.
 8. **Use CSS transitions** to animate a div's background color change smoothly.
 9. **Write a function** that removes all child elements from a `<div>` container efficiently.
 10. **Build a simple counter** that increments and decrements a number on button clicks.
 11. **Animate an element's opacity** using `setInterval()` and fade it out gradually.
 12. **Write a function** to store a user's preferred theme (dark/light) in `localStorage`.
 13. **Build a simple image slider** that switches images every 3 seconds using `setTimeout()`.
 14. **Create a dynamic table** where users can add new rows using a button click.
 15. **Use CSS transitions and JavaScript** to slide a div left or right when a button is clicked.
-

Expert Level (Optimizing DOM Updates, Event Delegation, Local Storage)

1. **Optimize a script** that updates 100 elements at once without slowing down the page.
2. **Implement event delegation** to handle clicks on dynamically added list items.
3. **Write a function** that saves an input field's value in `localStorage` and loads it when the page reloads.
4. **Modify the DOM structure** by moving an element from one container to another dynamically.
5. **Use the MutationObserver API** to detect and log changes in the DOM automatically.
6. **Use IntersectionObserver** to detect when an element enters the viewport.

7. **Implement infinite scrolling** by appending content when the user reaches the bottom of the page.
 8. **Detect and log changes in the DOM** using `MutationObserver`.
 9. **Optimize a script** that updates hundreds of elements at once without blocking the UI.
 10. **Write a function** that dynamically loads content from an API and inserts it into the DOM efficiently.
 11. **Use `IntersectionObserver`** to lazy-load images only when they appear in the viewport.
 12. **Optimize event listeners** by debouncing input field updates to reduce unnecessary function calls.
 13. **Create a function** that removes elements dynamically when they are clicked.
 14. **Write a script** that collapses or expands an FAQ section when a question is clicked.
 15. **Implement a progress bar** that fills up as the user scrolls down the page.
-

16. JavaScript execution flow, memory management, async behavior, and performance optimization questions covering advanced core concepts

1. Execution Context & Call Stack

1. Explain the **difference between global execution context and function execution context** with an example.
2. What will be the output of the following code, and why?

```
function first() {  
    console.log("First");  
}  
  
function second() {  
    first();  
    console.log("Second");  
}  
  
second();  
console.log("Third");
```

3. How does JavaScript manage **execution context** when a function calls itself recursively?

4. Explain the order of execution in the following code and predict the output:

```
function foo() {  
  console.log("foo");  
}  
function bar() {  
  foo();  
  console.log("bar");  
}  
setTimeout(bar, 0);  
console.log("baz");
```

5. What happens if a function has a very deep recursive call stack? How does JavaScript prevent a crash?

6. What will happen if a function inside another function returns `this`? Give an example.

7. What will be the output of the following code, and why?

```
function first() {  
  console.log("Inside first");  
  second();  
}  
function second() {  
  console.log("Inside second");  
}  
first();  
console.log("Outside functions");
```

8. What is tail call optimization in JavaScript, and when is it useful?

2. Hoisting & Temporal Dead Zone (TDZ)

1. What will happen when executing the following code?

```
console.log(a);  
var a = 5;  
console.log(b);  
let b = 10;
```

2. Explain the concept of **"Dead Zone"** in **let/const declarations** with an example.

3. What will be the output of the following code, and why?

```
console.log(a);  
let a = 10;
```

4. How does **hoisting** behave differently for `var`, `let`, and `const` variables?
5. Explain **why function expressions are not hoisted** like function declarations.
6. What will be the output of this code, and why?

```
console.log(typeof a);  
let a = 10;
```

7. **How does hoisting differ for function declarations and function expressions?** Explain with an example.
8. Modify the following code so that it **doesn't throw a `ReferenceError`** due to TDZ:

```
console.log(x);  
let x = 5;
```

3. Closures & Lexical Scope

1. Create a **closure-based function** that generates a unique ID each time it is called.
2. Explain **lexical scope** using an example with nested functions.
3. Modify the following code to ensure it correctly logs numbers from 1 to 5 with a delay of 1 second each:

```
for (var i = 1; i <= 5; i++) {  
  setTimeout(() => console.log(i), 1000);  
}
```

4. **Write a function that uses closures** to create a private counter that increments but can't be directly modified.
5. What is the output of this code, and why?

```
function outer() {  
  let count = 0;  
  return function inner() {  
    count++;  
    console.log(count);  
  };  
}  
  
const fn1 = outer();  
const fn2 = outer();  
  
fn1();  
fn1();  
fn2();
```

6. How does lexical scope affect the `this` keyword inside a closure? Give an example.

7. Write a function that generates a sequence of Fibonacci numbers using closures.

8. What will be the output of the following code, and why?

```
function outer() {  
  let count = 0;  
  return function inner() {  
    console.log(count++);  
  };  
}  
  
const counter1 = outer();  
const counter2 = outer();  
  
counter1();  
counter1();  
counter2();
```

9. How can closures be used to implement memoization?

4. Prototype & Prototypal Inheritance

1. What will be the output of the following code, and why?

```
function Person(name) {
  this.name = name;
}
Person.prototype.greet = function () {
  console.log(`Hello, ${this.name}`);
};
const user = new Person("Alice");
user.greet();
```

2. How can you **modify an existing prototype** of an object without affecting other instances?
3. What is the **difference between classical inheritance (used in Java, C#) and prototypal inheritance**?
4. **How can you prevent modifications** to an object's prototype?
5. **Write a prototype method** for an `Animal` constructor that all instances can use.
6. **Explain the difference between `Object.create()` and constructor functions** in JavaScript.
7. **What will be the output of this prototype-based inheritance code?**

```
function Parent() {
  this.name = "Parent";
}
Parent.prototype.sayHello = function () {
  return `Hello from ${this.name}`;
};
const child = new Parent();
console.log(child.sayHello());
```

8. How do you check if an object has a property of its own or inherited from the prototype?

5. this Keyword & Binding (call, apply, bind)

1. What will be the output of the following code?

```
const obj = {
  value: 10,
  method: function () {
    return this.value;
  },
};
console.log(obj.method());
const newMethod = obj.method;
console.log(newMethod());
```

2. Explain how **call**, **apply**, and **bind** work with an example.

3. Modify the following function using **.bind()** so that it correctly refers to **obj** when called:

```
const obj = { num: 42 };
function printNum() {
  console.log(this.num);
}
```

4. Predict the output of this code:

```
var name = "Global";
const person = {
  name: "Alice",
  greet: function () {
    console.log(this.name);
  },
};
setTimeout(person.greet, 1000);
```

5. How does explicit binding (**call**, **apply**, **bind**) affect function execution? Give an example.

6. Write a function using **bind()** that ensures the correct **this** context when used as a callback.

7. What will be the output of the following code?

```
const obj = {
  num: 5,
  multiply: function (factor) {
    return this.num * factor;
  },
};

const newMultiply = obj.multiply;
console.log(newMultiply(2));
```

8. Modify the above code using `.bind()` so that it correctly refers to `obj`.

6. Event Loop & Microtask Queue

1. Predict the output of the following code and explain why:

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("End");
```

2. What is the **priority order** of tasks in the event loop? Where do `setTimeout()`, `Promise.then()`, and `fetch()` fit?

3. Predict the output and explain why:

```
console.log("Start");
setTimeout(() => console.log("Timeout 1"), 1000);
setTimeout(() => console.log("Timeout 2"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("End");
```

4. How does the event loop differ in browser JavaScript vs Node.js?

5. Predict the output of the following async code execution:

```
console.log("Start");
setTimeout(() => console.log("Timeout 1"), 0);
Promise.resolve().then(() => console.log("Promise 1"));
setTimeout(() => console.log("Timeout 2"), 100);
console.log("End");
```

6. Explain why microtasks (Promises) run before macrotasks (`setTimeout`).

7. Garbage Collection (Mark & Sweep Algorithm)

1. What kind of **JavaScript** objects are eligible for garbage collection, and why?
 2. What is a **memory leak** in **JavaScript**, and how can you prevent it?
 3. Explain what happens to memory when a large object is set to `null`.
 4. How does **JavaScript** handle circular references in garbage collection?
 5. What happens when an object references itself in **JavaScript**?
 6. What is **WeakMap**, and how does it help in garbage collection?
-

8. Promises & Async/Await

1. Convert the following **callback-based function** into a `Promise`-based one:

```
function fetchData(callback) {  
  setTimeout(() => {  
    callback("Data received");  
  }, 2000);  
}
```

2. Write an **async function** that fetches data from a fake API using `fetch()` and handles errors with `try...catch`.

3. Convert the following **callback-based function** into a **Promise-based function**:

```
function fetchData(callback) {  
  setTimeout(() => {  
    callback("Data received");  
  }, 2000);  
}
```

4. What will be the output of the following **async function** execution?

```
async function test() {
  console.log("Start");
  await new Promise((resolve) => setTimeout(resolve, 2000));
  console.log("End");
}
test();
console.log("Outside function");
```

5. What happens if an `await` statement inside an `async` function fails? How do you handle errors properly?

9. Generators & Iterators

1. Write a generator function that yields even numbers indefinitely.

2. What will be the output of the following generator-based code?

```
function* generatorExample() {
  yield "First";
  yield "Second";
  return "Third";
}
const gen = generatorExample();
console.log(gen.next());
console.log(gen.next());
console.log(gen.next());
```

Here are **20 more JavaScript questions**, including **real-world applications** and **FAANG-level interview challenges** covering **core JS concepts** [🔗](#)

1. Execution Context & Call Stack

1. **(FAANG Interview)** Explain the output of this JavaScript execution flow:

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
```

2. **(Real-World)** You are developing a browser-based code editor. How can you measure and limit stack depth to prevent call stack overflow?

2. Hoisting & Temporal Dead Zone (TDZ)

3. **(FAANG Interview)** Why does the following code throw an error?

```
function test() {  
  console.log(x);  
  let x = 5;  
}  
  
test();
```

4. **(Real-World)** You are refactoring old `var` declarations in a legacy codebase to `let` and `const`. What precautions should you take?

3. Closures & Lexical Scope

5. **(FAANG Interview)** What is a closure? How does the following function behave?

```
function counter() {  
  let count = 0;  
  return () => ++count;  
}  
  
const increment = counter();  
console.log(increment());  
console.log(increment());
```

6. **(Real-World)** You need to debounce a search bar input. How would you implement this using closures?

4. Prototype & Prototypal Inheritance

7. **(FAANG Interview)** Given the following code, what will be logged?

```
function Animal(name) {  
  this.name = name;  
}  
  
Animal.prototype.speak = function () {  
  return `${this.name} makes a noise`;  
};  
  
const cat = new Animal("Whiskers");  
console.log(cat.speak());
```

8. **(Real-World)** How can you create an object that shares properties from two prototypes (e.g., a `FlyingCar` that inherits from both `Car` and `Airplane`)?

5. this Keyword & Binding (call, apply, bind)

9. **(FAANG Interview)** Fix the following snippet so that it correctly logs "Hello John":

```
const person = {
  name: "John",
  greet: function () {
    setTimeout(function () {
      console.log(`Hello ${this.name}`);
    }, 1000);
  },
};
person.greet();
```

10. **(Real-World)** When implementing an event listener in a UI library, why is it important to use `.bind(this)`?

6. Event Loop & Microtask Queue

11. **(FAANG Interview)** What is the output of the following?

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("End");
```

12. **(Real-World)** In a high-performance web app, how can you ensure that a computationally expensive task does not block the UI thread?

7. Garbage Collection (Mark & Sweep Algorithm)

13. **(FAANG Interview)** Explain why the following code causes a memory leak:

```
let cache = {};
function storeData(key, value) {
  cache[key] = value;
}
```

14. **(Real-World)** How would you use `WeakMap` to prevent memory leaks in an event-driven architecture?

8. Promises & Async/Await

15. **(FAANG Interview)** Rewrite the below callback-based function to use Promises and `async/await`:

```
function getData(callback) {
  setTimeout(() => {
    callback("Data retrieved");
  }, 2000);
}
```

16. **(Real-World)** You are building an API that fetches user details from multiple endpoints. How can you use `Promise.all()` to optimize network requests?

9. Generators & Iterators

17. **(FAANG Interview)** Implement a generator function that produces an infinite sequence of Fibonacci numbers.

18. **(Real-World)** How would you create a custom iterable object for paginated API requests?

10. Memoization & Caching

19. **(FAANG Interview)** Implement a memoized version of the following function:

```
function slowFunction(n) {
  return n * 2; // Simulating a slow operation
}
```

20. **(Real-World)** How can you use memoization to optimize results in an autocomplete search field?

1. Execution Context & Call Stack

1. **(FAANG Interview)** What will be the output of this JavaScript snippet, and why?

```
function first() {  
  console.log("First");  
}  
  
function second() {  
  first();  
  console.log("Second");  
}  
  
function third() {  
  second();  
  console.log("Third");  
}  
  
third();
```

2. **(Real-World)** Your application crashes due to **"Maximum call stack size exceeded."** What are the possible causes, and how can you prevent this?
-

2. Hoisting & Temporal Dead Zone (TDZ)

3. **(FAANG Interview)** Explain why the following code logs `undefined` instead of throwing an error:

```
console.log(foo);  
var foo = 10;
```

4. **(Real-World)** You are working on a team project where developers mix `var`, `let`, and `const`. What guidelines would you set to avoid hoisting-related issues?
-

3. Closures & Lexical Scope

5. **(FAANG Interview)** Implement a function `createCounter` that maintains a private counter using closures and returns functions to increment and get the value.
6. **(Real-World)** You need to create a **rate limiter** that allows only **one API call per second**. How would you achieve this using closures?
-

4. Prototype & Prototypal Inheritance

7. **(FAANG Interview)** Modify the following prototype-based inheritance code so that `SuperHero` correctly inherits from `Person`:

```
function Person(name) {
  this.name = name;
}

function SuperHero(name, power) {
  this.power = power;
}
```

8. **(Real-World)** When working with **third-party libraries**, how can you **extend prototype functions safely** without breaking existing code?
-

5. this Keyword & Binding (call, apply, bind)

9. **(FAANG Interview)** Rewrite this function using `bind()` to correctly log the person's name:

```
const person = {
  name: "Alice",
  greet: function () {
    setTimeout(function () {
      console.log(`Hello, ${this.name}`);
    }, 1000);
  },
};

person.greet();
```

10. **(Real-World)** How can `Function.prototype.apply()` be used to flatten an array efficiently?
-

6. Event Loop & Microtask Queue

11. **(FAANG Interview)** What will be the output of the following code?

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
new Promise((resolve) => resolve("Promise")).then(console.log);
console.log("End");
```

12. **(Real-World)** How can you use the **Event Loop** to **optimize animations in a game engine**?
-

7. Garbage Collection (Mark & Sweep Algorithm)

13. **(FAANG Interview)** Why does this code cause a **memory leak**?

```
let obj = {};  
let ref = obj;  
obj = null;
```

14. **(Real-World)** How can `WeakMap` be used to prevent memory leaks in a large-scale event-driven application?
-

8. Promises & Async/Await

15. **(FAANG Interview)** Convert this **callback-based** function to use `async/await`:

```
function fetchData(callback) {  
  setTimeout(() => {  
    callback("Data received");  
  }, 1000);  
}
```

16. **(Real-World)** You are developing a **highly scalable API**. How can you use `Promise.race()` to implement a timeout mechanism?
-

9. Generators & Iterators

17. **(FAANG Interview)** Implement a generator function that produces an infinite sequence of **prime numbers**.
18. **(Real-World)** How can a **custom iterator** be useful for paginated API requests?
-

10. Memoization & Caching

19. **(FAANG Interview)** Implement a **memoized Fibonacci function** in JavaScript.
20. **(Real-World)** How can memoization be used to improve the performance of a **spell checker or autocomplete system**?
-

Here are **20 fresh JavaScript questions** covering the **Advanced Part 1 topics** you listed, from **basic to advanced**, ensuring no repetition. ☐

1. Type of Scopes (Global, Function, Block, Module)

1. **(Basic)** What is the difference between **global scope**, **function scope**, and **block scope** in JavaScript? Provide an example.
2. **(Basic)** Predict the output and explain the scope behavior:

```
let x = 10;
function test() {
    console.log(x);
    let x = 20;
}
test();
```

3. **(Intermediate)** How does **module scope** work in JavaScript ES6+? Explain using an example with `import` and `export`.
4. **(Intermediate)** Given the following code, which variables are accessible inside the `printValue` function?

```
const a = 5;
function outer() {
    let b = 10;
    if (true) {
        let c = 20;
        function printValue() {
            console.log(a, b, c);
        }
    }
}
```

5. **(Advanced)** How do **IIFE (Immediately Invoked Function Expressions)** help in avoiding global scope pollution? Write an example.

2. Execution Context & Call Stack

6. **(Basic)** What are the **three phases** of JavaScript execution context?
7. **(Basic)** Explain the **Call Stack** and why this code results in a **stack overflow**:

```
function recursive() {
    recursive();
}
recursive();
```

8. **(Intermediate)** What happens when the following script runs? Explain step by step.

```
console.log(a);
var a = 10;
function foo() {
  console.log(b);
  var b = 20;
}
foo();
```

9. **(Intermediate)** Modify the following code so that it correctly maintains the execution order using `setTimeout()`:

```
console.log("Start");
setTimeout(() => console.log("Middle"), 0);
console.log("End");
```

10. **(Advanced)** How does **tail call optimization (TCO)** work, and why is it useful in execution contexts?
-

3. Hoisting & Temporal Dead Zone (TDZ)

11. **(Basic)** Define **hoisting** and explain how it affects variables declared with `var`, `let`, and `const`.
12. **(Basic)** What will be the output of the following code, and why?

```
console.log(a);
var a = 10;
console.log(b);
let b = 20;
```

13. **(Intermediate)** Explain the **Temporal Dead Zone (TDZ)** in JavaScript and why the following code throws an error:

```
console.log(x);
let x = 5;
```

14. **(Intermediate)** In terms of **hoisting**, why is function expression behavior different from function declaration? Explain with an example.
15. **(Advanced)** How does hoisting affect **arrow functions** when stored in variables? Write an example to demonstrate the behavior.
-

4. Closures & Lexical Scope

16. **(Basic)** Explain **closures** and provide an example where they are useful.
17. **(Basic)** Why does the following function retain access to `x` even after `outer()` has finished executing?


```
function outer() {  
  let x = 10;  
  return function inner() {  
    console.log(x);  
  };  
}  
  
const fn = outer();  
fn();
```

18. **(Intermediate)** Implement a function `createMultiplier` that returns a function to **multiply a number** by a given factor using closures.
19. **(Advanced)** How can closures be used to implement a **private variable** in JavaScript? Provide an example.
20. **(Advanced)** Given the following function, what will be the output and why?

```
function counter() {  
  let count = 0;  
  return function () {  
    console.log(count++);  
  };  
}  
  
const c1 = counter();  
c1();  
c1();  
  
const c2 = counter();  
c2();  
c2();
```

20 JavaScript Advanced questions covering Advanced Part 1 topics:

1. Module Pattern (ES6 Modules & CommonJS)

1. **(Basic)** What is the main difference between **ES6 Modules** and **CommonJS**?
2. **(Basic)** How would you **export multiple functions** from an **ES6 module**?
3. **(Intermediate)** Convert the following **CommonJS module** to an **ES6 module**:

```
const greet = () => console.log("Hello");  
module.exports = { greet };
```

4. **(Intermediate)** Explain why `import` statements are **hoisted**, but `require()` is not.
 5. **(Advanced)** How can you create a **private variable** inside a module that cannot be modified from the outside?
-

2. Factory & Singleton Pattern

6. **(Basic)** What is the **Factory Pattern**, and when should you use it?
7. **(Basic)** Implement a simple **Factory function** that creates different types of **shapes** (`Circle`, `Square`).
8. **(Intermediate)** Explain the **Singleton Pattern** and provide a simple JavaScript implementation.
9. **(Intermediate)** Modify the following **Singleton** to be **lazy-loaded** (only created when needed):

```
class Logger {
  constructor() {
    console.log("Logger Initialized");
  }
}

const loggerInstance = new Logger();
export default loggerInstance;
```

10. **(Advanced)** What are the **downsides of the Singleton Pattern**, and how can we avoid its limitations in **large-scale applications**?
-

3. Observer Pattern (Event Emitters)

11. **(Basic)** What is the **Observer Pattern**, and where is it commonly used?
 12. **(Basic)** Implement a **simple event emitter** in JavaScript that allows adding and emitting events.
 13. **(Intermediate)** Modify the event emitter to support **removing listeners** for specific events.
 14. **(Intermediate)** How can you use the Observer Pattern to implement **real-time chat notifications**?
 15. **(Advanced)** Compare **Observer Pattern vs. Pub/Sub Pattern**. Which one scales better in a **distributed microservices architecture**?
-

4. Decorator Pattern (HOFs & Middleware)

16. **(Basic)** What is a **Higher-Order Function (HOF)**? Provide an example.
 17. **(Basic)** Implement a simple **logger decorator function** that logs input and output of any function.
 18. **(Intermediate)** How is the **Decorator Pattern** used in **Express.js middleware**?
 19. **(Advanced)** Implement a **rate-limiting middleware** using a decorator function.
 20. **(Advanced)** How can decorators be used to **implement role-based access control (RBAC)** in a web application?
-

5. Proxy & Reflect API

21. **(Basic)** What is a **JavaScript Proxy**, and how does it work?
 22. **(Basic)** Create a **Proxy** that logs all **get and set operations** on an object.
 23. **(Intermediate)** Explain the difference between `Proxy` and `Object.defineProperty()`.
 24. **(Advanced)** Implement a **validation proxy** that prevents setting negative values for an object property.
 25. **(Advanced)** How can the **Reflect API** be used to simplify working with Proxies?
-

1. Module Pattern (ES6 Modules & CommonJS)

1. **(Basic)** Why do ES6 modules use **strict mode** by default?
 2. **(Intermediate)** How can you dynamically **import** a module in ES6? Provide an example.
 3. **(Intermediate)** What is **tree shaking**, and how does it work in ES6 modules?
 4. **(Advanced)** How does bundling tools like **Webpack** handle **ES6 modules vs. CommonJS**?
 5. **(Advanced)** Explain how **circular dependencies** happen in ES6 modules and ways to **resolve them**.
-

2. Factory & Singleton Pattern

6. **(Basic)** How does the Factory Pattern promote **code reusability**?
7. **(Intermediate)** Modify a **Factory function** to support **caching** previously created objects.
8. **(Intermediate)** Explain the difference between **Factory Pattern** and **Constructor Functions** in JavaScript.
9. **(Advanced)** Convert the following **Factory function** into a **class-based Singleton**:

```
function DatabaseConnection() {  
    return { connect: () => console.log("Connected!") };  
}
```

10. **(Advanced)** Why should we avoid modifying a **Singleton instance** after creation, and how can we enforce immutability?
-

3. Observer Pattern (Event Emitters)

11. **(Basic)** What are the key components of the **Observer Pattern**?
 12. **(Intermediate)** Implement an event emitter that supports **asynchronous event handling**.
 13. **(Intermediate)** How can you prevent **memory leaks** when using the Observer Pattern in JavaScript?
 14. **(Advanced)** How would you implement a **custom Redux-like store** using the Observer Pattern?
 15. **(Advanced)** Compare **WebSockets vs. Observer Pattern** for handling real-time data updates.
-

4. Decorator Pattern (HOFs & Middleware)

16. **(Basic)** How do decorators improve **code readability** in JavaScript?
17. **(Intermediate)** Implement a **memoization decorator** for optimizing function calls.
18. **(Intermediate)** Modify a function decorator to support **multiple decorators in sequence**.

19. **(Advanced)** How does the **Decorator Pattern** help in designing a **plugin system** for applications?
 20. **(Advanced)** Implement an **audit logger decorator** that logs function execution details to a database.
-

5. Proxy & Reflect API

21. **(Basic)** Explain how **Proxy traps** work and list **three commonly used traps**.
 22. **(Intermediate)** Implement a **proxy** that prevents adding **new properties** to an object.
 23. **(Intermediate)** How does `Reflect` simplify working with **JavaScript Proxies**?
 24. **(Advanced)** Implement a **Proxy-based security layer** that restricts access to object properties based on user roles.
 25. **(Advanced)** How can **Proxies** be used to track **performance bottlenecks** in function execution?
-

1. Module Pattern (ES6 Modules & CommonJS)

1. **(Basic)** What are the benefits of using **named exports** over **default exports** in ES6 modules?
 2. **(Intermediate)** How can you create a **module alias** in a project using Webpack or Node.js?
 3. **(Intermediate)** How do you handle **circular dependencies** in ES6 modules?
 4. **(Advanced)** Implement a **lazy-loaded module** that only imports when required.
 5. **(Advanced)** How does **module caching** work in CommonJS, and how can you reset a cached module?
-

2. Factory & Singleton Pattern

6. **(Basic)** What is the key advantage of **Factory Pattern** over **Constructor Functions**?
 7. **(Intermediate)** Implement a Factory function that returns **different database connections** based on input.
 8. **(Intermediate)** Explain how the **Singleton Pattern** can cause issues in unit testing and how to avoid them.
 9. **(Advanced)** Implement a Singleton that **automatically destroys itself** if it is not used for a period of time.
 10. **(Advanced)** How can the Singleton pattern be combined with the **Factory pattern** in JavaScript?
-

3. Observer Pattern (Event Emitters)

11. **(Basic)** How can you **unsubscribe** from an event in an **Event Emitter**?
 12. **(Intermediate)** Implement a **throttled event emitter** that limits the frequency of event emissions.
 13. **(Intermediate)** What is the difference between the **Observer Pattern** and **Promises**?
 14. **(Advanced)** Implement an event system where **listeners execute only once** after subscribing.
 15. **(Advanced)** How would you optimize an Observer-based **notification system** in a large-scale application?
-

4. Decorator Pattern (HOFs & Middleware)

16. **(Basic)** Explain how **higher-order functions (HOFs)** can act as decorators.
17. **(Intermediate)** Implement a **time-tracking decorator** that logs function execution time.
18. **(Intermediate)** How does the **Decorator Pattern** apply to **React Higher-Order Components (HOCs)**?

- 19. **(Advanced)** Implement a decorator that **logs API calls** and caches responses.
 - 20. **(Advanced)** How would you implement **A/B testing** in an application using decorators?
-

5. Proxy & Reflect API

- 21. **(Basic)** How does the Proxy API help in **object validation**?
 - 22. **(Intermediate)** Implement a **Proxy wrapper** that restricts setting object properties to a predefined list.
 - 23. **(Intermediate)** How can **Reflect API** simplify **Proxy handler methods**?
 - 24. **(Advanced)** Implement a Proxy-based **Internationalization (i18n) system** that automatically translates strings.
 - 25. **(Advanced)** How can Proxies be used to **implement an auto-save feature** in forms?
-